MeshFlight Part 0 Plan

Foundation phase plan for contracts, scaffolding, first vertical slice, and clean integration into the rest of the project

Document type	Implementation planning memo
Project phase	Part 0: framing, contracts, scaffolding, and first vertical slice
Primary outcome	A working pipeline from scenario authoring to simulation to live visualization, with contracts stable enough that ns-3, RL, telemetry, and validation can be added without rewrites
Source basis	Built from the uploaded MeshFlight design specification, especially the sections on architecture, environment model, data contracts, build plan, and recommended stack

North-star for Part 0: do not start with RL training, perfect radio realism, or production observability. Start with the architecture that makes those later pieces plug in cleanly: shared contracts, repo structure, scenario compiler, simulator stub, control plane, and live operator UI.

1. Executive Summary

Part 0 is the phase where the project becomes a real system instead of a loose set of ideas. The goal is to choose the stack, lock the data contracts, scaffold the repo, define the boundaries between services, and ship one narrow but complete vertical slice: map authoring, scenario save and load, compile, simulation run, event stream, and live visualization.

At the end of Part 0, you should be able to open a local operator console, place nodes and obstacles on a 2D map, save a scenario, compile it into simulator-ready assets, start a run, watch nodes and links update over time, inject a failure, and see the baseline controller react. The system does not need RL yet. It does need clean interfaces, deterministic artifacts, and enough rigor that later phases can be integrated without structural rewrites.

2. Objectives, Non-Goals, and Definition of Done

Area	What must be true by the end of Part 0	What is explicitly out of scope for Part 0
Architecture	Subsystems and data boundaries are written down and reflected in the repository structure. Shared schemas are versioned and consumed by	No microservice sprawl and no premature split into many independently deployed services.

Area	What must be true by the end of Part 0	What is explicitly out of scope for Part 0
	both backend and frontend.	
World authoring	A user can create, edit, save, and reload a 2D scenario with obstacles, drones, gateways, clients, and scheduled events.	No polished design tool, terrain engine, or 3D editor.
Simulation	A lightweight simulator can run fixed-step or event-driven updates over a compiled scenario, generate snapshots, and emit runtime events.	No full ns-3-backed packet fidelity yet.
Control plane	There is a simple API for scenario compile, run lifecycle, and streaming updates to the UI.	No distributed control plane, auth system, or multi-user support.
Observability	Run IDs, logs, snapshots, and metrics files are persisted in a predictable layout.	No full Prometheus, Grafana, Tempo stack unless time remains.
AI / policy	Deterministic baselines exist for routing and basic reconnect-first movement.	No RL training or policy optimization in this phase.

3. Recommended Stack for Part 0

Use the final project stack where it already helps, but defer the expensive pieces that will slow down iteration. The priority in Part 0 is fast development with strong contracts and clear replacement points.

Layer	Part 0 choice	Why this is the right choice now	Replacement or extension path
Frontend	React + TypeScript + Vite + React Flow + canvas overlay	Fastest route to an operator console with topology editing, inspectors, and live updates.	Can stay in place for the whole project.
Backend / control plane	Python 3.11 + FastAPI + Pydantic + WebSockets	Keeps compiler, simulator orchestration, and future RL stack in one language.	Optional gRPC service layer can be added later without replacing FastAPI immediately.

Layer	Part 0 choice	Why this is the right choice now	Replacement or extension path
Scenario schema	JSON files validated with Pydantic and JSON Schema	Human-readable, easy to diff, easy to save from UI, and easy to archive as artifacts.	Add protobuf only for streaming/runtime contracts later.
Compiler	Python service module with Shapely, NumPy, NetworkX	Good for geometry, graph prep, and precomputing scenario assets.	Can remain permanent even after ns-3 is introduced.
Simulator core	Python lightweight simulator stub	Lets you validate architecture before paying the complexity cost of ns-3 integration.	Replace or wrap with ns-3 adapter in Phase 2.
Artifacts	Local filesystem first with stable object-style directory layout	Removes deployment friction while preserving future MinIO compatibility.	Point same paths at MinIO or S3-compatible storage later.
Developer environment	Docker Compose for infra, local venv/node tooling for app iteration	Fast local startup and easier debugging.	Expand to Kubernetes only when validation or scaling demands it.

4. Integration Principles That Should Guide Everything

- Contract-first, not UI-first and not RL-first. The scenario schema, runtime event schema, and artifact layout are the backbone of the whole project.
- Stable boundaries over clever shortcuts. Every module should have one obvious responsibility and should communicate through typed data models rather than ad hoc dictionaries.
- Deterministic runs wherever possible. Each scenario and run should carry a seed, schema version, and config snapshot.
- Replaceability by design. The simulator stub should be easy to swap with an ns-3-backed engine because the outside-facing contract stays the same.
- Observable from day one. Even before full telemetry infrastructure, each run should write events, snapshots, summaries, and logs in a stable format.
- One vertical slice beats ten half-finished subsystems. A thin end-to-end pipeline is much more valuable than isolated components that do not connect.

5. Repository Structure and Ownership Boundaries

Use a monorepo. You are building editor, compiler, simulator, APIs, and later RL. Keeping shared contracts in one repository avoids drift and makes integration easier.

```
meshflight/
  apps/
    editor-ui/                  # React operator console and environment editor
    api/                        # FastAPI entrypoint for compile, run, replay, and
streaming
  services/
    scenario_compiler/          # Authoring input -> normalized scenario assets
    sim_core/                   # Lightweight simulator stub and later ns-3 adapter
boundary
    replay_service/             # Reads snapshots/events and serves replay timelines
    baseline_policies/          # Deterministic routing and reconnect-first controllers
  packages/
    schema/                     # JSON schema, Pydantic models, shared enums, versioning
    runtime_contracts/          # Event and snapshot contracts; protobuf can live here
later
    frontend_types/             # Generated TS types or shared type definitions
  artifacts/
    scenarios/                  # Saved scenario sources and compiled variants
    runs/                       # events, snapshots, metrics, logs, summaries by run_id
    reports/                    # benchmark notes, screenshots, and exported docs
  infra/
    docker/                     # compose files and service config
    prometheus/                 # reserved for later
    grafana/                    # reserved for later
  docs/
    architecture/               # diagrams, ADRs, integration notes
    api/                        # endpoint and event contract docs
    runbooks/                   # local setup, troubleshooting, workflows
  scripts/                      # helpers for setup, validation, artifact cleanup
  tests/
    schema/
    compiler/
    sim/
    api/
    e2e/
```

Critical rule: do not let shared contracts live inside only the backend or only the frontend. Put them in a neutral package and generate or mirror types outward from there.

6. Contracts You Should Lock in Part 0

6.1 Scenario source contract

This is the file written by the editor and loaded back into the editor. It should be human-readable, schema-versioned, and close to the authored intent. Keep it declarative rather than storing simulator internals.

- Metadata: scenario_id, schema_version, title, description, seed, created_at, authoring_version.
- Map: width, height, grid resolution, origin, layers, optional presets.
- Entities: drones, gateways, clients, interference emitters, demand zones, no-fly zones.
- Obstacles: buildings, walls, vegetation zones, with shape data and semantics like attenuation and no-fly behavior.
- Traffic classes and schedules: source zones, destination nodes, rates, SLAs, event timing.
- Chaos events: node failure, obstacle appearance, interference spike, demand burst.

6.2 Compiled scenario contract

This is not the same as the source scenario. It is the normalized, simulator-ready output produced by the compiler. Keep it explicit so that the simulator never has to guess what the editor meant.

- Normalized coordinates and IDs with all defaults resolved.
- Precomputed obstacle indices and line-of-sight hints.
- Candidate graph edges based on communication range and coarse penalty models.
- Waypoint candidates, mobility bounds, and per-drone movement constraints.
- Workload schedule expanded into time-indexed or event-indexed runtime entries.
- Compilation metadata: source hash, compiler version, config hash, seed, and generated_at timestamp.

6.3 Runtime event contract

The UI needs low-latency updates. Define a typed event stream now even if you use WebSockets first and protobuf later.

Event type	Purpose	Required payload ideas
node_state_changed	Update node position, battery, role, or status	node_id, time, x, y, battery, queue depth, status
link_state_changed	Update connectivity and link quality	src, dst, connected, snr/cost/capacity
flow_started / flow_ended	Track flow lifecycle	flow_id, class, src, dst, rate, SLA
flow_rerouted	Explain self-healing or baseline response	flow_id, old_path, new_path, reason
failure_injected	Mark deterministic chaos events	failure_type, target, start_time
policy_action_chosen	Make later policy behavior legible	policy_name, action, affected_nodes
snapshot_emitted	Anchor replay and scrubbing	snapshot_id, time, summary metrics

6.4 Snapshot and artifact contract

Every run should create a predictable output layout. This matters for replay, debugging, and later benchmark automation.

Artifact	What it should contain
artifacts/runs/<run_id>/events.jsonl	Append-only runtime event stream
artifacts/runs/<run_id>/snapshots/	Periodic state snapshots, ideally JSON first and Parquet later

Artifact	What it should contain
artifacts/runs/<run_id>/metrics.json	Summary metrics and time-windowed stats
artifacts/runs/<run_id>/config.json	Exact resolved configuration used for the run
artifacts/runs/<run_id>/logs.txt	Structured logs or newline-delimited log file
artifacts/runs/<run_id>/summary.md	Human-readable run summary and notable events

7. Part 0 Staged Plan

Stage 0A: local setup and repo scaffolding

Create the monorepo, establish Python and Node environments, add formatting and linting tools, create the shared schema package, and make sure apps can import contracts from one place. Do not write product logic before this skeleton exists.

Stage 0B: scenario schema and validation

Design and validate the source scenario schema and compiled scenario schema. Write examples by hand before building the editor so you know what a good scenario object looks like.

Stage 0C: editor MVP

Implement a narrow 2D authoring tool with pan, zoom, grid, entity placement, drag, selection, basic property editing, and save/load.

Stage 0D: compiler MVP

Convert authored scenarios into normalized simulator-ready assets and write them into the artifact structure. This is the first place where geometry and graph logic become real.

Stage 0E: simulator MVP

Run a lightweight simulation over the compiled scenario with deterministic baselines, event emission, and snapshot output.

Stage 0F: control plane and streaming

Add API endpoints to compile scenarios, start runs, stop runs, fetch run state, and stream updates to the operator UI.

Stage 0G: first end-to-end demo and hardening

Run a saved scenario from the editor through compile and simulation, visualize it live, inject a failure, capture artifacts, and fix whatever integration pain this exposes.

8. Detailed Work Breakdown, Deliverables, and Success Criteria

8.1 Stage 0A - local setup and repo scaffolding

Need to code / create

- Monorepo root with apps, services, packages, artifacts, docs, and tests directories.
- Python project setup for API, compiler, and simulator modules.
- Node project setup for editor-ui.
- Common formatting, linting, and type-checking commands.
- Bootstrap run scripts so one command can start the API and one command can start the UI.
- Architecture notes document that names module responsibilities and future replacement points.

Success looks like

- A new developer can clone the repo and bring up the UI and API locally without guessing hidden steps.
- Shared schema package can be imported by both backend and frontend.
- There is one canonical place for environment variables and one canonical local-start workflow.

Integration guardrails

- Do not let environment-specific values leak into code. Use config files and env vars.
- Do not duplicate type definitions across frontend and backend.

8.2 Stage 0B - scenario schema and validation

Need to code / create

- Pydantic models for source scenario, compiled scenario, runtime event, snapshot, and run config.
- JSON Schema exports or generated schema files.
- Validation tests with both valid and intentionally invalid scenarios.
- Two to three canonical example scenarios that will later become seeded benchmark fixtures.

Success looks like

- Saving an invalid scenario fails with clear error messages.
- Example scenarios validate cleanly and remain backward-compatible across small revisions.
- The schema is specific enough that the compiler does not need vague fallback behavior.

Integration guardrails

- Add schema_version now, not later.
- Keep authored intent separate from compiled simulator internals.

8.3 Stage 0C - editor MVP

Need to code / create

- 2D canvas with pan, zoom, and optional grid snap.
- Palette for building, wall, tree zone, drone, gateway, client, demand zone, and event placement.
- Selection model and inspector panel for editing properties.
- Save/load flow for scenario JSON.
- Visual layer toggles for labels, communication radius, and obstacle shading.

Success looks like

- You can author a usable scenario in minutes without touching raw JSON.
- Reloaded scenarios look the same as when they were saved.
- The editor model maps directly onto the scenario schema instead of maintaining a hidden incompatible representation.

Integration guardrails

- The editor should not compute simulator logic on its own.
- Keep editor state serializable and schema-aligned.

8.4 Stage 0D - compiler MVP

Need to code / create

- Normalization pass for IDs, defaults, map bounds, and entity references.
- Obstacle indexing and coarse line-of-sight / attenuation precompute.
- Candidate connectivity graph builder.
- Waypoint candidate generation and mobility constraint packaging.
- Compiled output writer plus compilation report summary.

Success looks like

- Running compile on the same source scenario and same config produces deterministic compiled output.
- Compiler failures point to the specific malformed object or reference.
- The compiled output can be consumed by the simulator without any editor-only assumptions.

Integration guardrails

- Treat compiler output as an interface artifact, not an internal temporary object.
- Include compiler version and source hash in output metadata.

8.5 Stage 0E - simulator MVP

Need to code / create

- Run lifecycle manager with create, start, stop, and tick loop.
- Node state model with position, battery, queue/load placeholders, and status.
- Link update logic using distance and simple obstacle penalties.

- Baseline routing controller and reconnect-first movement controller.
- Event emitter and periodic snapshot writer.
- Failure injection hooks for node loss and interference spike.

Success looks like

- For a small scenario, the run produces understandable topology changes over time.
- When a deterministic failure is injected, routing or movement baseline behavior changes in a way you can explain.
- Artifacts are written under one stable run_id with no missing pieces.

Integration guardrails

- Keep simulator internals behind a clean adapter boundary so ns-3 can slot in later.
- Separate engine state from presentation payloads.

8.6 Stage 0F - control plane and streaming

Need to code / create

- Endpoints to save scenario, compile scenario, list scenarios, start run, stop run, inspect run, and fetch artifacts.
- WebSocket stream for runtime events and optional periodic summary snapshots.
- Run status model with pending, compiling, ready, running, completed, failed states.
- Basic replay fetch endpoint that reads stored snapshots/events.

Success looks like

- The editor can trigger compile and run through the API instead of shell scripts.
- The UI receives live updates with consistent event shape and ordering.
- Finished runs can be reopened and replayed later.

Integration guardrails

- Design event payloads as versioned contracts.
- Do not make the frontend dependent on private simulator objects.

8.7 Stage 0G - end-to-end demo and hardening

Need to code / create

- One polished demo scenario with a gateway, multiple drones, obstacles, and one failure event.
- A scripted demo path: author or load scenario, compile, run, inject failure, inspect reroute or repositioning.
- E2E tests for save/load, compile, run, and artifact presence.
- Debugging checklists and runbook notes based on the first real integration pain points.

Success looks like

- You can record a short walkthrough that clearly shows the system working end to end.

- You trust the repository structure and contracts enough to begin Phase 1 and Phase 2 without a redesign.
- The top integration bugs are documented and mostly eliminated.

Integration guardrails

- Keep one canonical demo scenario as a regression fixture.
- Capture every important design choice in docs before memory drifts.

9. How to Set Up the System So Integration Stays Easy

Area	Setup decision	Reason it saves pain later
Configuration	Use one typed config layer per service and commit example env files for local dev.	Prevents mystery environment drift and makes runs reproducible.
IDs and versioning	Give every scenario, compiled artifact, and run a stable ID plus schema and config version metadata.	Lets replay, comparisons, and migrations work cleanly.
Artifact paths	Adopt object-style directory paths from the beginning even if stored locally.	Makes transition to MinIO or S3 mostly a storage backend change.
Type sharing	Generate or derive frontend types from backend schema definitions.	Prevents silent contract skew.
API boundaries	Expose only DTO-style payloads from the API, never raw simulator objects.	Lets backend internals evolve without breaking the UI.
Fixtures	Maintain small seeded scenarios that always run in CI or local regression scripts.	Gives you a stable benchmark and catches breakage early.
Testing	Test schemas, compiler, simulator, and e2e pipeline separately.	Pinpoints failures faster than one giant integration test.
Documentation	Write ADRs and runbooks as you go, not after the fact.	Future you will need them when ns-3 and RL are added.

10. Recommended Test Strategy for Part 0

Schema tests

- Valid scenario passes.
- Malformed coordinates, unknown references, and missing fields fail with clear messages.

- Schema version is required and covered by tests.

Compiler tests

- Compiled output is deterministic for fixed input and seed.
- Obstacle indexing and candidate graph generation behave as expected on small fixtures.
- Compiler metadata includes source hash and compiler version.

Simulator tests

- A tiny graph scenario produces expected connectivity changes.
- Failure injection changes output in a deterministic way.
- Run artifacts are written in the expected directory layout.

API tests

- Compile and run endpoints return stable payloads.
- WebSocket event stream emits valid event types in the right shape.
- Replay endpoints return stored data without special casing.

End-to-end tests

- Load fixture scenario from editor or API.
- Compile.
- Start run.
- Receive events.
- Confirm artifacts exist and can be reopened.

11. Exit Criteria for Part 0

Category	Exit criterion
Contracts	Scenario, compiled scenario, runtime event, snapshot, and artifact layouts are written down, versioned, and tested.
Editor	You can create, save, load, and edit a scenario with at least the core entities and obstacles.
Compiler	A saved scenario can be compiled into deterministic simulator-ready assets.
Simulator	A compiled scenario can be run and produce snapshots, events, and baseline reactions to failures.
Control plane	The UI can trigger compile and run flows through APIs and receive live updates.
Artifacts	Runs write stable, replayable output under a run_id.

Category	Exit criterion
Integration confidence	You can explain exactly where ns-3, RL, telemetry, and real-world validation will plug in without changing the public contracts.

12. Immediate Next Step After This Plan

The very first implementation move should be Stage 0A and Stage 0B together: scaffold the repository, create the shared schema package, and hand-author two example scenarios before building the editor. That gives you the data model, tests, and repository shape that everything else will plug into.

Prepared as a planning document for the MeshFlight project foundation phase.